

Proyecto 1

Taller de Aprendizaje Automático

Valentina Alaniz, Estéfano Bargas, Betina Cerecetto

Facultad de Ingeniería, Universidad de la República, Montevideo, Uruguay

Mayo 2024

Índice

1. Introducción	2
2. Análisis de datos	2
2.1. Análisis de datos faltantes	2
2.2. Análisis de distribución de los datos	2
3. Procesamiento de datos	3
4. Modelos considerados	4
4.1. Modelos lineales, SVM y Random Forest	4
4.2. Boosting	5
4.2.1. XGBoost	5
4.3. Red neuronal	6
4.3.1. Shap Values	7
5. Conclusión	7
A. Anexo	8
A.1. Oversampling	8
A.1.1. Random Forest	8

1. Introducción

El problema se trata de un experimento realizado en el Gran Colisionador de Hadrones, con el objetivo de detectar una partícula denominada “Bosón de Higgs”. Durante el experimento se provoca la colisión de un gran numero de partículas (cada colisión es denominada *evento*) y se obtienen valores para 30 características diferentes por cada una. Se requiere construir un clasificador que detecte si un evento provoco la presencia del Boson de Higgs (en cuyo caso se le llama *signal*) o no (*background*). El problema es de entrenamiento supervisado ya que tenemos un conjunto de datos previamente clasificado.

2. Análisis de datos

2.1. Análisis de datos faltantes

Para el análisis de datos, se investigó la composición del conjunto de prueba. Se identificó inicialmente una gran cantidad de datos faltantes en ciertas categorías, como se puede observar en la [Figura 1](#).

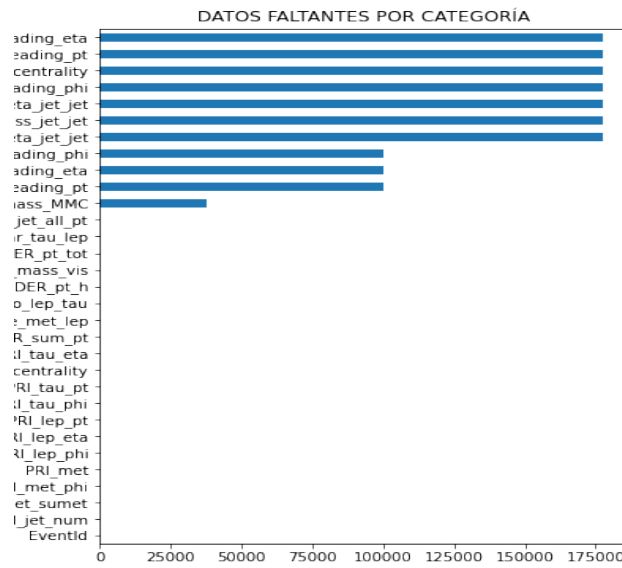


Figura 1: Datos faltantes por categoría

2.2. Análisis de distribución de los datos

En la segunda fase, se analizó como se distribuyen los datos faltantes dentro del conjunto. Se identificaron distintos tipos de distribución entre los datos de cada categoría. En primer lugar se encontraron distribuciones “clásicas” como la uniforme, la normal y la exponencial (Power Law). Estas distribuciones se encuentran referenciadas como [Figura 2](#), [Figura 3](#) y [Figura 4](#) respectivamente.

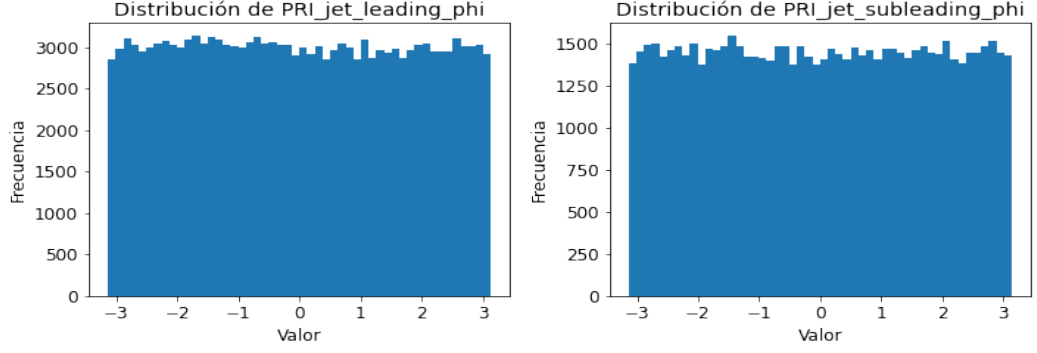


Figura 2: Distribución de datos uniforme

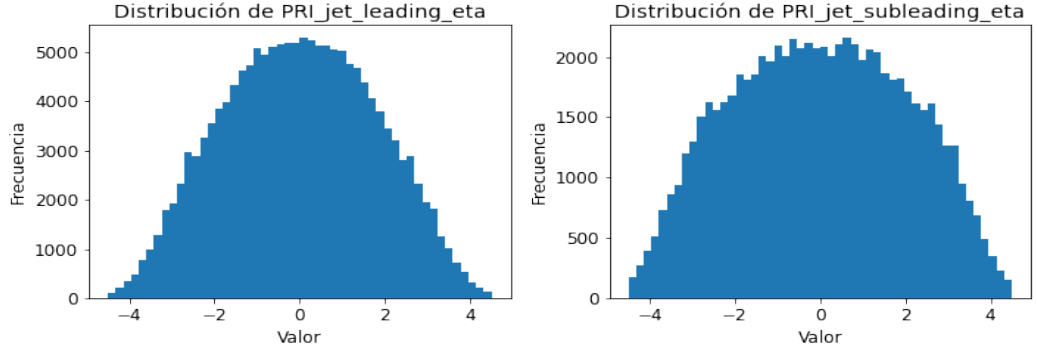


Figura 3: Distribución de datos normal

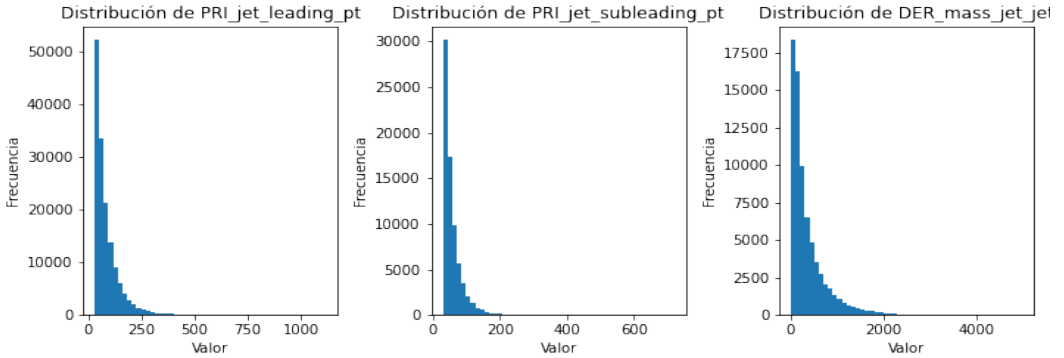


Figura 4: Distribución de datos exponencial

Además, se identificaron distribuciones de datos que no se ajustan a las fórmulas clásicas. Un ejemplo de esto es la característica “DER_lep_eta_centrality”, la misma muestra una distribución donde tiene probabilidades altas de ser cero, una probabilidad menor a ser exactamente uno, y muy pocas probabilidades de que los valores estén entre estos dos números. Es decir, la mayor densidad de valores es uno o cero como se puede observar en la [Figura 5](#).

Para la característica “DER_mass_MMC”, se observa que la distribución de los datos es normal sesgada. Esto se puede apreciar en la [Figura 6](#)

3. Procesamiento de datos

Como se mencionó en la sección anterior, las características dentro del conjunto de datos de entrenamiento presentan diferentes tipos de distribuciones. Para cada una, se implementó un pipeline de preprocesamiento adecuado para cada una.

Para las características que se distribuyen en bins como “DER_lep_eta_centrality”, se utilizó inicialmente un imputador personalizado llamado *FillWithPrevious* para tratar con los valores faltantes y un escalador de dos “bins”.

En el caso de las distribuciones normales, se empleó un imputador *SimpleImputer* y seguido de un escalador *StandardScaler* empleando como estrategia la mediana, con el objetivo de estandarizar las variables.

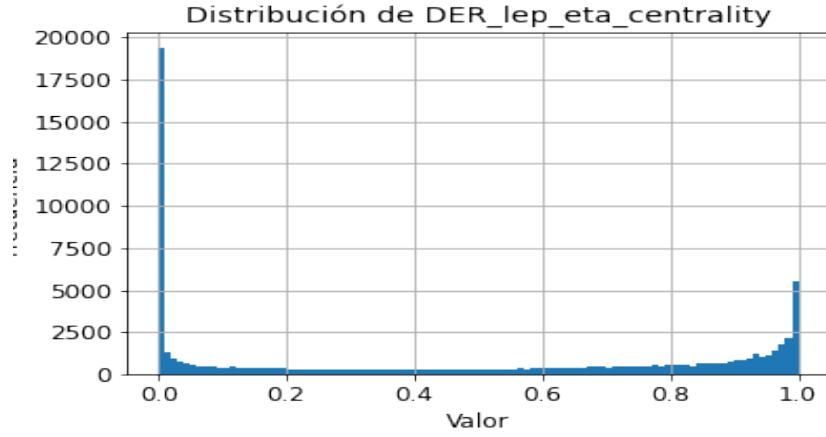


Figura 5: Distribución de datos de DER lep eta centrality

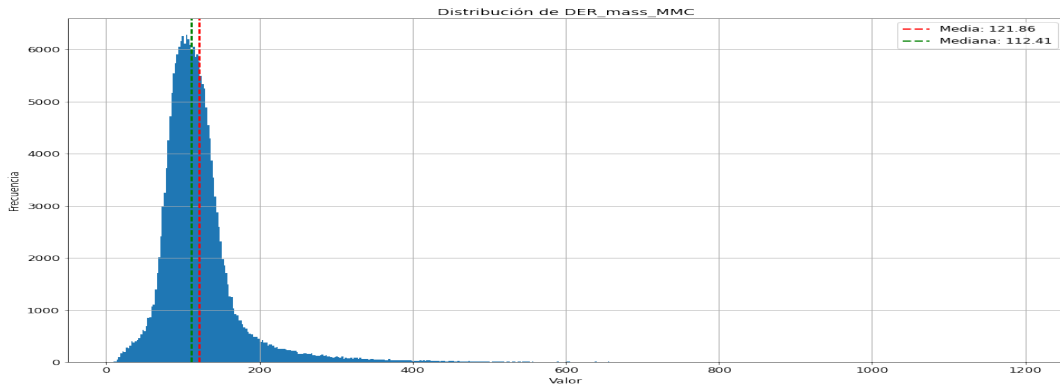


Figura 6: Distribución en el conjunto de datos DER mass MMC

Para las distribuciones tipo *PowerLaw*, se aplicó también un *SimpleImputer* y posteriormente se un *FunctionTransformation* transformando los datos mediante un logaritmo con el fin de normalizar los datos. Es importante tener en cuenta que los atributos con distribución tipo *PowerLaw* no incluyen valores negativos. Además la función *FunctionTransformation* aplica $\log(x + 1)$, por lo que no habría problemas con los valores nulos. Con esta información es posible aplicar la transformación de manera segura.

Finalmente, para el resto de características, se aplicó un *SimpleImputer* para el manejo de los valores faltantes y posteriormente se aplicó un *StandardScaler* para normalizar los datos.

4. Modelos considerados

4.1. Modelos lineales, SVM y Random Forest

Se comienza con la fase de entrenamiento explorando inicialmente modelos lineales: regresión lineal y logística. Posteriormente, se avanza a modelos mas complejos: SVM (con kernel RBF) y Random Forest. Durante el entrenamiento, se considera el peso de cada instancia ya que a pesar de que introduce un desbalance significativo, es necesario tomarlo en cuenta para generalizar mejor.

El preprocesamiento de los datos se llevó a cabo utilizando el pipeline previamente descrito, distribuyendo el 80 % de los datos para entrenamiento, 20 % para validación. La tabla [Tabla 1](#) compara el rendimiento de los modelos en esta primera etapa. Los valores corresponden al threshold que maximizan la métrica F1.

Para *Random Forest* se implementó una búsqueda aleatoria de hiperparametros, ajustando variables como la cantidad de estimadores y la profundidad máxima de cada árbol.

Los modelos lineales mostraron un subajuste significativo a los datos, clasificando la gran mayoría de las instancias del conjunto de validación como “background”. Este comportamiento es probablemente consecuencia del gran desbalance de clases y la complejidad insuficiente de los modelos para capturar la función objetivo.

El modelo *SVM* a pesar de tener mayor complejidad al utilizar un kernel *RBF* tampoco logra ajustarse a los datos, clasificando todas las instancias como “background”. Además, este tipo de modelos no se desempeñan bien en

Modelo	Precisión	Recall
Regresión lineal	0 %	0 %
Regresión logística	14 %	0.07 %
SVM (RBF)	0 %	0 %
Random forest	88 %	84 %

Tabla 1: Comparación entre modelos sencillos.

términos de eficiencia en conjuntos de entrenamiento muy grandes, ya que es muy lento el entrenamiento.

En contraste con lo anterior, *Random Forest* demostró un rendimiento considerablemente mayor en el conjunto de validación. Su *AMS Score* fue de 1.505, mostrando una mayor capacidad de generalización. Este resultado nos motivó a probar con *Boosting* en *Random Forest*. También se explorarán modelos de redes neuronales.

Vale aclarar que para este método también se realizó el experimento de balancear el conjunto de datos mediante *oversampling*, los resultados se pueden ver en el Anexo:A.1.1 donde se detallan la comparación de los resultados obtenidos,

4.2. Boosting

Como se mencionó anteriormente para mejorar el rendimiento de los modelos se plantea utilizar técnicas de *Boosting*. En este contexto puede llegar a ser de utilidad ya que son métodos efectivos al enfrentarse a conjuntos de datos desbalanceados, especialmente ayudan a identificar la clase minoritaria.

Para abordar la solución, se implementaron dos formas de *Boosting*: *XGBoost* y *AdaBoost*. Este enfoque puede mejorar el rendimiento ya que ajusta iterativamente el peso de las instancias mal clasificadas en los pasos anteriores, haciendo que el modelo se enfoque en los casos más difíciles en las rondas siguientes. Con este enfoque no sólo se aumenta la precisión global del modelo sino también capacidad de clasificar correctamente los “signal”. En la siguiente sección se profundizará sobre los resultados obtenidos aplicando *XGBoost*, si bien los resultados con *AdaBoost* fueron buenos con respecto a los modelos lineales, no logró superar el rendimiento de *XGBoost*.

4.2.1. XGBoost

Se empleó la técnica de *Gradient Boosting* utilizando la librería de *XGBoost*. Siguiendo el mismo esquema de preprocesamiento de los modelos anteriores. Para la selección óptima de hiperparámetros, se utilizó una búsqueda aleatoria (*Random Search*).

El modelo *XGBoost* se configuró inicialmente con 1000 estimadores, utilizando *Early Stopping* para prevenir el sobreajuste y minimizar los tiempos de entrenamiento. Se utilizó la métrica *AUC* para medir el rendimiento del modelo durante el entrenamiento, y la métrica *AMS* para la evaluación durante el *Random Search*.

Este modelo fue probado sobre el conjunto de entrenamiento sólo preprocesado, con y sin *oversampling*. Se puede observar que el rendimiento del modelo mejora significativamente con la adición de *Oversampling*.

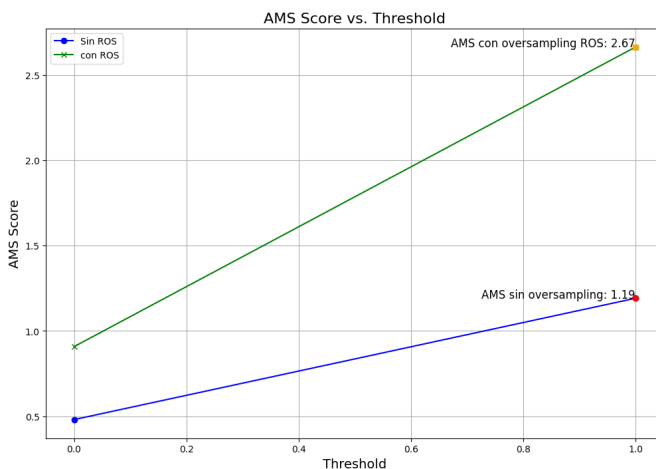


Figura 7: AMS vs threshold con y sin ROS

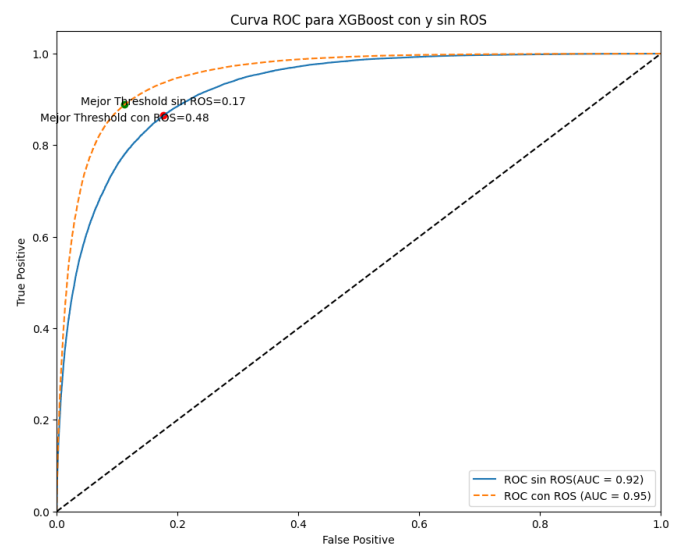


Figura 8: Curva ROC con y sin ROS

Se pueden observar los experimentos de [XGBoost sin oversampling](#).

4.3. Red neuronal

Se utilizó la biblioteca *Keras* para entrenar una red neuronal y ajustar sus hiperparametros. Se utilizó el mismo preprocesamiento que para los modelos anteriores, con la diferencia de una normalización como ultimo paso antes de entrar a la red. Se utilizó el algoritmo Hyperband para hallar la forma de la red que minimiza el error de entropia cruzada en un conjunto de validación (separado de los datos de entrenamiento), ajustando la cantidad de capas ocultas (con un maximo de 8), la cantidad de neuronas en cada capa (con un maximo de 50) y el learning rate del optimizador. Como funcion de activacion para las capas ocultas se eligio *ReLU* y para la capa de salida *Sigmoid* (ya que predecimos la probabilidad de que un dato sea signal). Se ejecuto el entrenamiento final con 200 épocas.

Capa oculta	Neuronas
1	60
2	33
3	53
4	64
5	42

Tabla 2: Forma final de la red neuronal encontrada.

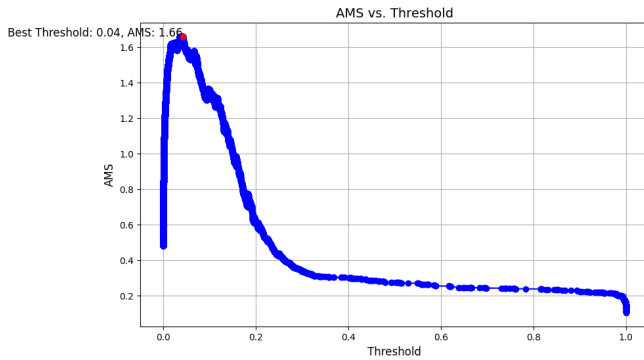


Figura 9: AMS en función del threshold para la red neuronal entrenada.

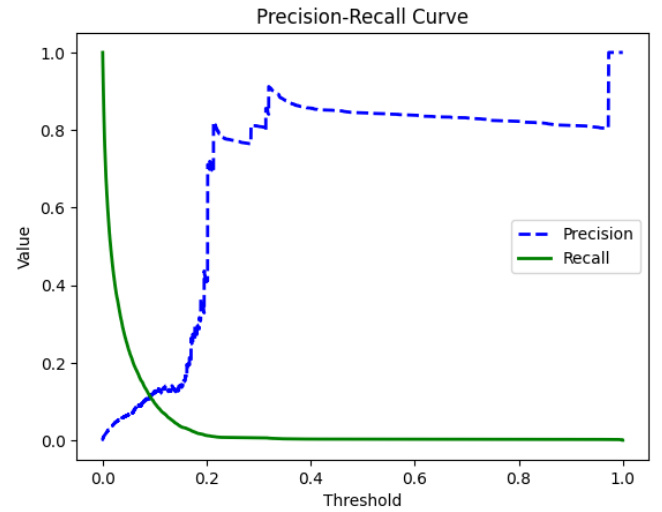


Figura 10: Precisión y recall en función del threshold para la red neuronal entrenada.

Luego de varios intentos y muchas horas de ejecución, la mejor red que se encontró tiene la forma presentada en la [Tabla 2](#). En la [Figura 10](#) se puede ver las curvas de precisión y recall en función al threshold y en la [Figura 9](#) el AMS en función del threshold, se eligió el punto de operación que maximiza la métrica AMS en validación. Finalmente, se aplicó *Oversampling* a este método, pero no se obtuvo una diferencia significativa en su rendimiento. [Se puede acceder al experimento de evaluación para observar mas métricas de rendimiento.](#)

4.3.1. Shap Values

Se genero una representación gráfica de la importancia relativa de las 10 características mas importantes utilizando la librería *SHAP*. Un *shap value* negativo representa un impacto que disminuye la salida (mas probabilidad de que la instancia pertenezca a la clase background) mientras que un valor positivo indica un aumento en la salida (mas probabilidad de que sea un evento signal). Algunas características (como las dos primeras) tienen una influencia directa (a mayor valor, mayor probabilidad) mientras que otras (como la tercera) influyen de manera inversa.

Estos datos pueden resultar muy útiles para el análisis físico del experimento en casos de eventos interesantes (aquellos catalogados como signal).

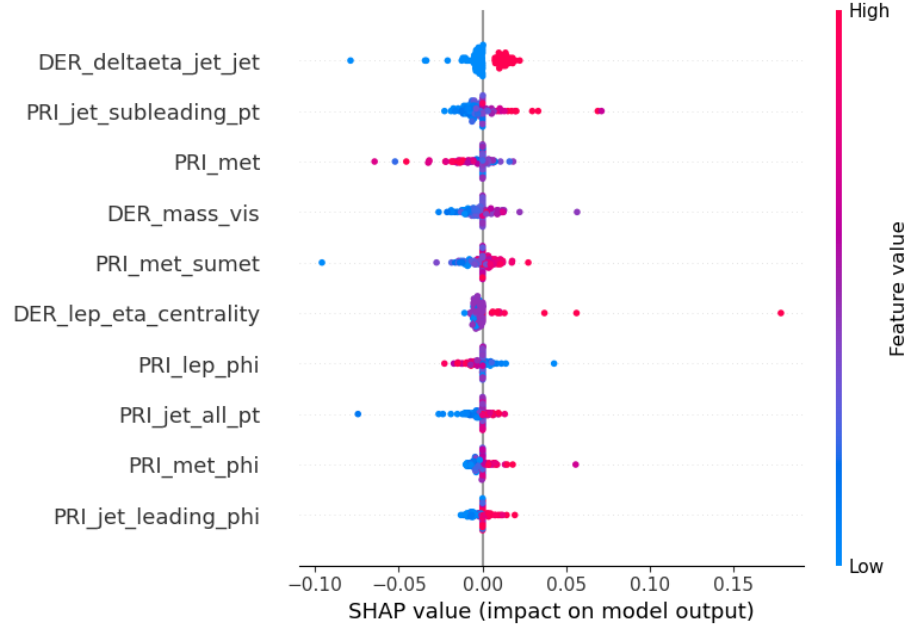


Figura 11: SHAP *summary plot* para la red neuronal entrenada.

5. Conclusión

Los mejores modelos obtenidos son el de XGBoost y la red neuronal. Luego de evaluar varias métricas se eligió XGBoost con oversampling como mejor modelo en primera instancia por tener una métrica AMS mayor en nuestro conjunto de validación. Sin embargo, al evaluar el modelo en el conjunto de test, los resultados obtenidos (AMS 3.35) fueron significativamente diferentes a los encontrados en el conjunto de validación (AMS de 3.32). Esta diferencia podrían indicar que la distribución o naturaleza de los datos en el conjunto de test es distinta a la del conjunto de validación, lo que sugiere que el conjunto de validación apartado durante el entrenamiento no refleja correctamente la capacidad de generalización del modelo y resultamos en un modelo sobreajustado. Por lo tanto en segunda instancia se eligió como mejor modelo a la red neuronal, ya que eligiendo el punto de operación con el mismo criterio, los resultados superaron a los de XGBoost en test (AMS 3.41), pero no en validación (AMS 1.61).

A. Anexo

A.1. Oversampling

El conjunto de train que nos brinda kaggle viene de por si desbalanceado, esto significa que hay mas muestras de una clase que de otra. En este caso tenemos que los datos signals son el 34.3% de la cantidad de datos con la que vamos a entrenar(Figura 12)

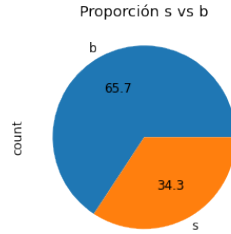


Figura 12: Desbalance en el conjunto de datos

Para balancear el conjunto de datos se utilizó RandomOversampler el cual genera nuevos datos mediante la elección aleatoria de datos que ya existen en el conjunto de train. Es decir no "inventa" datos nuevos solo aumenta la densidad de datos de la clase minoritaria, en este caso signal.

A.1.1. Random Forest

Para este experimento se utilizó el mismo preprocesamiento que en los otros modelos y como algoritmo de aprendizaje se utilizó Random Forest. A su vez se utilizó la búsqueda de hyperparametros mediante el uso de RandomizedSearchCV. Este experimento se realizó con y sin *oversampling*, sus resultados se pueden apreciar en Tabla 3 Como podemos ver

Modelo	Precisión	Recall	AMS en validación
Random forest	88 %	84 %	1.505
Random forest con Over Sampling	87 %	79 %	2.168

Tabla 3: Comparación entre modelo Random Forest con y sin Over Sampling.

en la tabla el modelo utilizando *oversampling* tiene un mejor AMS Score. Lamentablemente al evaluarlo en test se obtuvo un valor de 2.27 siendo este valor mucho menor al AMS score en test del modelo elegido. Atribuimos esta diferencia a que el modelo se sobreajustó a los datos y por eso tiene una baja performance en datos de test.